Especificação formal do SM83 e validação de um emulador de Game Boy

Enzo Lopes Campanholo [1] Felipe Bonatto Zwaizdis Scaquetti [1]
Guilherme Itihara [1]

[1]Faculdade de Computação e Informática (FCI)
Universidade Presbiteriana Mackenzie São Paulo, SP – Brasil

2026

Orientador: André Kishimoto

Palavras-chave: especificação executável; verificação formal; Sail; Rocq; emulação de CPU; teste diferencial.

1 Introdução

Uma arquitetura de conjunto de instruções, ou ISA (do inglês instruction set architecture), é o contrato entre o software e o processador. Ela define os registradores, as instruções e o efeito delas sobre o estado da máquina. Em geral, esse contrato existe apenas na forma de manuais escritos em linguagem natural ou pseudocódigo, e cada emulador, compilador, ferramenta e programador precisa interpretar o texto por conta própria. Para reduzir essa ambiguidade, alguns fabricantes e pesquisadores passaram a escrever especificações formais e executáveis de ISAs como ARM, RISC-V e x86 (ARMSTRONG et al., 2019; DASGUPTA et al., 2019; REID, 2016). Contudo, dado o esforço herculeano de descrever, e às vezes provar, as especificações, nem todos os processadores contam com especificações formais. O Game Boy, lançado pela Nintendo em 1989, usa uma CPU de 8 bits chamada SM83 que, embora tenha sido objeto de estudo de hobistas por pelo menos duas décadas, não conta com uma especificação formal oficial publicamente disponível. Seu comportamento está descrito em documentação mantida pela comunidade, construída por observação, engenharia reversa e estudo empírico (GBDEV, 2026; JAVANAINEN, 2026). O SM83, embora lembre outros processadores como o Intel 8080 e o Zilog Z80, difere em aspectos importantes (JAVANAINEN, 2026). A culminação do estudo do SM83 está nos emuladores. Cada emulador oferece a sua interpretação da ISA do SM83 e, quando dois emuladores discordam, não há um modelo formal oficial de referência que, por si só, permita decidir qual está certo. O problema deste trabalho é examinar em que medida uma

especificação formal executável do SM83 pode sustentar provas formais, servir como parte de um emulador e apoiar uma comparação sistemática entre diferentes implementações.

O objetivo geral é construir e avaliar um modelo executável do SM83, escrito em Sail, que una especificação formal, provas em Rocq, emulação de jogos e testes comparativos reproduzíveis. Os objetivos específicos são: (i) descrever em Sail o estado da CPU, as instruções do Game Boy original (DMG-01) e a ordem dos eventos observáveis a cada ciclo de máquina; (ii) provar em Rocq um conjunto delimitado de propriedades do modelo, como a decodificação sem ambiguidade e o cálculo das flags em uma família de instruções aritméticas; (iii) integrar a CPU gerada a partir do modelo a um emulador de Game Boy com vídeo, controles e som; (iv) comparar execuções do modelo e do emulador com pelo menos três emuladores independentes entre si; (v) guardar e reduzir casos de divergência para permitir sua análise e repetição; e (vi) relatar resultados, hipóteses e limites sem confundir testes com provas.

2 Referencial Teórico

Esta seção apresenta os conceitos que sustentam o trabalho na ordem em que eles se ligam. Primeiro vem a CPU a ser modelada. Depois, as especificações formais de ISAs e as ferramentas escolhidas para escrevê-las e prová-las. Por fim, a emulação e os testes que comparam o modelo com outras implementações.

2.1 O processador SM83

O SM83 trabalha com oito registradores de 8 bits (A, F, B, C, D, E, H e L), que também podem ser usados em pares de 16 bits (AF, BC, DE e HL), além do ponteiro de pilha (SP) e do contador de programa (PC). O registrador F guarda quatro flags nos bits 7 a 4 (zero, subtração, half-carry e carry), e cada instrução define quais delas altera. A tabela principal de instruções tem 256 códigos. Onze deles são inválidos e travam a CPU, e o código CB dá acesso a uma segunda tabela de 256 instruções de rotação, deslocamento e manipulação de bits (GBDEV, 2026). No Game Boy original, o relógio principal é de 4,194304 MHz. Cada ciclo de máquina, ou M-cycle, corresponde a quatro períodos desse relógio, e as instruções gastam de um a seis M-cycles (GBDEV, 2026; JAVANAINEN, 2026).

Alguns comportamentos só fazem sentido quando se observa a ordem dos eventos no tempo. A instrução EI, por exemplo, só habilita as interrupções depois da instrução seguinte. Já o chamado HALT bug ocorre quando a CPU executa HALT com as interrupções globalmente desabilitadas (IME igual a zero), mas com uma interrupção habilitada no registrador IE e pendente no registrador IF. Nesse caso, o contador de programa não é incrementado na busca do próximo opcode, o que pode fazer o byte desse opcode ser lido novamente (GBDEV, 2026). Por isso, este trabalho usa o M-cycle como unidade de avanço do modelo. A cada passo, o modelo expõe uma leitura, uma escrita ou um ciclo sem acesso à memória, o que permite ligá-lo ao restante do emulador sem descer ao nível dos sinais elétricos.

2.2 Especificações formais e executáveis de ISAs

Uma especificação formal descreve um sistema em uma linguagem com significado matemático preciso, o que permite provar propriedades sobre ele. Backus (1978) já argumentava que programas escritos em um estilo com leis algébricas simples são mais fáceis

de analisar do que programas presos ao modelo de von Neumann. Quando a especificação também pode ser executada, ela se torna um programa de referência. Ela pode rodar testes, ser comparada com o hardware e servir de base para provas. Essa combinação já foi aplicada a ISAs grandes. Goel, Hunt Jr. e Kaufmann (2017) construíram em ACL2 um simulador formal da ISA x86 e o validaram contra processadores reais. Dasgupta et al. (2019) definiram no framework K uma semântica formal das instruções de usuário do x86-64. Wang et al. (2020) formalizaram a ISA SPARCv8 diretamente em Coq. Na ARM, Reid (2016) relata como a empresa tornou executável a especificação que aparece nos seus manuais e passou a usá-la na verificação formal de processadores.

Uma especificação executável, porém, pode conter erros. As provas feitas sobre ela garantem propriedades do modelo, mas não, por si só, sua fidelidade ao hardware. Reid (2017) trata esse problema ao validar a especificação da ARM v8-M contra um conjunto de propriedades de alto nível escritas de forma independente. No caso do SM83, a questão pesa mais, pois não há especificação oficial. O modelo deste trabalho terá de ser confrontado com a documentação técnica (GBDEV, 2026; JAVANAINEN, 2026), com suítes de teste e com outros emuladores, sem supor de antemão que algum deles está certo.

2.3 Sail e Rocq

Sail é uma linguagem para descrever a semântica de ISAs. Sua escrita lembra o pseudocódigo dos manuais de fabricantes, mas tem tipos e significado definidos com precisão (ARMSTRONG et al., 2019). A partir de um só modelo, as ferramentas do Sail geram emuladores em C e OCaml e definições para os assistentes de provas Coq, Isabelle e HOL4. Já existem modelos em Sail de ARMv8-A, RISC-V e CHERI-MIPS, e o modelo de RISC-V foi adotado como referência pela organização que mantém essa arquitetura (ARMSTRONG et al., 2019; REMS PROJECT, 2026). Esse duplo caminho motiva a escolha do Sail, pois o mesmo texto alimenta as provas e a CPU do emulador. Isso não garante, contudo, que o código C gerado preserve cada detalhe das definições usadas nas provas. As provas tratam das definições formais, e os testes dão evidência sobre o executável.

O Rocq, chamado Coq antes da versão 9.0, de 2025 (ROCQ DEVELOPMENT TEAM, 2025), é um assistente de provas interativo. O usuário escreve definições, enunciados e provas, e o sistema só aceita uma prova quando consegue verificar cada passo de forma mecânica. Com Coq, Leroy (2009a, 2009b) provou que o compilador CompCert preserva a semântica dos programas que compila, e Sammler et al. (2022), na ferramenta Islaris, combinaram modelos Sail com a lógica de separação Iris para verificar código de máquina. Antes de tentar provar uma propriedade, vale testá-la. O QuickChick leva testes baseados em propriedades ao Coq, e Paraskevopoulou et al. (2015) mostram como verificar formalmente que esses testes avaliam o que se pretende. Neste trabalho, as provas cobrirão um conjunto delimitado de propriedades, escolhido após um ensaio inicial com as definições geradas pelo Sail.

2.4 Emulação de CPU

Um emulador reproduz em software o comportamento de outra máquina. A forma mais simples é o interpretador, que busca, decodifica e executa uma instrução por vez, e o emulador em C gerado pelo Sail segue essa forma. O QEMU ganha velocidade com tradução binária dinâmica, que converte blocos de código da máquina emulada em instruções da máquina hospedeira e reaproveita a tradução nas execuções seguintes (BELLARD, 2005). Outra linha gera simuladores rápidos a partir de descrições de arquitetura. O Pydgin usa

compiladores JIT por meta-tracing para isso (LOCKHART; ILBEYI; BATTEN, 2015), e o Pydrofoil aplica a mesma técnica a modelos Sail. Em testes com RISC-V, seus autores relatam ganhos de mais de 230 vezes sobre o C gerado pelo Sail (BOLZ-TEREICK et al., 2025). O SM83 executa pouco mais de um milhão de M-cycles por segundo, um ritmo modesto para um computador atual. Mesmo assim, o emulador precisa manter vídeo e som em tempo real, e esses trabalhos indicam caminhos caso o código gerado se mostre lento.

2.5 Teste diferencial e análise de divergências

Como as provas não cobrem todo o modelo e o modelo não cobre todo o Game Boy, os testes seguem necessários. O teste diferencial executa a mesma entrada em duas ou mais implementações de um mesmo comportamento e trata qualquer diferença como sinal de erro em ao menos uma delas (MCKEEMAN, 1998). Martignoni et al. (2009) aplicaram essa ideia a emuladores de CPU x86, comparando-os com um processador físico. O EXAMINER faz o mesmo para ARM, com casos gerados a partir da especificação oficial (JIANG et al., 2022), e Campbell e Stark (2016) extraem regras de comportamento de um modelo executável para gerar testes.

Uma diferença, porém, não revela qual lado erra. Esse é o problema do oráculo de testes (BARR et al., 2015), e ele impede tratar o modelo formal como correto só por ser formal. Para investigar uma diferença, ajuda reduzir o caso a uma entrada menor que ainda a produz. O delta debugging automatiza essa redução, sem garantir um mínimo global (ZELLER; HILDEBRANDT, 2002), e Regehr et al. (2012) mostram que, em programas, os cortes precisam respeitar a estrutura do código para não gerar entradas inválidas. No SM83, remover uma instrução pode mudar os endereços de desvio, então o redutor precisa conhecer o formato do programa, e não só seus bytes.

Essas linhas se completam neste trabalho. O Sail fornece a especificação executável do SM83. O Rocq verifica propriedades escolhidas dessa especificação. O código C gerado a partir dela é a CPU do emulador. E o teste diferencial, com redução de casos, compara o modelo e o emulador com outras implementações e ajuda a localizar erros em qualquer um dos lados.

Referências

ARMSTRONG, A. et al. ISA semantics for ARMv8-A, RISC-V, and CHERI-MIPS. *Proc. ACM Program. Lang.*, Association for Computing Machinery, New York, NY, USA, v. 3, n. POPL, p. 71:1–71:31, jan. 2019. Disponível em: <https://doi.org/10.1145/3290384>.

BACKUS, J. Can programming be liberated from the von Neumann style? a functional style and its algebra of programs. *Commun. ACM*, Association for Computing Machinery, New York, NY, USA, v. 21, n. 8, p. 613–641, ago. 1978. ISSN 0001-0782. Disponível em: <https://doi.org/10.1145/359576.359579>.

BARR, E. T. et al. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, v. 41, n. 5, p. 507–525, 2015. Disponível em: <https://doi.org/10.1109/TSE.2014.2372785>.

BELLARD, F. QEMU, a fast and portable dynamic translator. In: *2005 USENIX Annual Technical Conference (USENIX ATC 05)*. Anaheim, CA: USENIX Association, 2005. Disponível em: <https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator>.

BOLZ-TEREICK, C. F. et al. Pydrofoil: Accelerating Sail-Based Instruction Set Simulators. In: ALDRICH, J.; SILVA, A. (Ed.). *39th European Conference on Object-Oriented Programming (ECOOP 2025)*. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. (Leibniz International Proceedings in Informatics (LIPIcs), v. 333), p. 3:1–3:31. ISBN 978-3-95977-373-7. ISSN 1868-8969. Disponível em: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2025.3>.

CAMPBELL, B.; STARK, I. Extracting behaviour from an executable instruction set model. In: *2016 Formal Methods in Computer-Aided Design (FMCAD)*. IEEE, 2016. p. 33–40. Disponível em: <https://doi.org/10.1109/FMCAD.2016.7886658>.

DASGUPTA, S. et al. A complete formal semantics of x86-64 user-level instruction set architecture. In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. New York, NY, USA: Association for Computing Machinery, 2019. (PLDI 2019), p. 1133–1148. ISBN 9781450367127. Disponível em: <https://doi.org/10.1145/3314221.3314601>.

GBDEV. *Pan Docs*. 2026. Disponível em: <https://gbdev.io/pandocs/>. Acesso em: 27 set. 2026.

GOEL, S.; HUNT JR., W. A.; KAUFMANN, M. Engineering a formal, executable x86 ISA simulator for software verification. In: HINCHEY, M.; BOWEN, J. P.; OLDEROG, E.-R. (Ed.). *Provably Correct Systems*. Cham: Springer International Publishing, 2017. p. 173–209. ISBN 978-3-319-48628-4. Disponível em: <https://doi.org/10.1007/978-3-319-48628-4_8>.

JAVANAINEN, J. *GB: complete technical reference*. [S.l.], 2026. Revisão 192, 16 ago. 2026. Disponível em: <https://gekkio.fi/files/gb-docs/gbctr.pdf>. Acesso em: 27 set. 2026.

JIANG, M. et al. EXAMINER: automatically locating inconsistent instructions between real devices and CPU emulators for ARM. In: *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. New York, NY, USA: Association for Computing Machinery, 2022. (ASPLOS '22), p. 846–858. ISBN 9781450392051. Disponível em: <https://doi.org/10.1145/3503222.3507736>.

LEROY, X. Formal verification of a realistic compiler. *Commun. ACM*, Association for Computing Machinery, New York, NY, USA, v. 52, n. 7, p. 107–115, jul. 2009. ISSN 0001-0782. Disponível em: <https://doi.org/10.1145/1538788.1538814>.

LEROY, X. A formally verified compiler back-end. *Journal of Automated Reasoning*, v. 43, n. 4, p. 363–446, dez. 2009. ISSN 1573-0670. Disponível em: <https://doi.org/10.1007/s10817-009-9155-4>.

LOCKHART, D.; ILBEYI, B.; BATTEN, C. Pydgin: generating fast instruction set simulators from simple architecture descriptions with meta-tracing JIT compilers. In: *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2015. p. 256–267. Disponível em: <https://doi.org/10.1109/ISPASS.2015.7095811>.

MARTIGNONI, L. et al. Testing CPU emulators. In: *Proceedings of the Eighteenth International Symposium on Software Testing and Analysis*. New York, NY, USA: Association for Computing Machinery, 2009. (ISSTA '09), p. 261–272. ISBN 9781605583389. Disponível em: <https://doi.org/10.1145/1572272.1572303>.

MCKEEMAN, W. M. Differential testing for software. *Digital Technical Journal*, v. 10, n. 1, p. 100–107, 1998.

PARASKEVOPOULOU, Z. et al. Foundational property-based testing. In: URBAN, C.; ZHANG, X. (Ed.). *Interactive Theorem Proving*. Cham: Springer International Publishing, 2015. p. 325–343. ISBN 978-3-319-22102-1. Disponível em: <https://doi.org/10.1007/978-3-319-22102-1_22>.

REGEHR, J. et al. Test-case reduction for C compiler bugs. In: *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*. New York, NY, USA: Association for Computing Machinery, 2012. (PLDI '12), p. 335–346. ISBN 9781450312059. Disponível em: <https://doi.org/10.1145/2254064.2254104>.

REID, A. Trustworthy specifications of ARM® v8-A and v8-M system level architecture. In: *2016 Formal Methods in Computer-Aided Design (FMCAD)*. IEEE, 2016. p. 161–168. Disponível em: <https://doi.org/10.1109/FMCAD.2016.7886675>.

REID, A. Who guards the guards? formal validation of the Arm v8-M architecture specification. *Proc. ACM Program. Lang.*, Association for Computing Machinery, New York, NY, USA, v. 1, n. OOPSLA, p. 88:1–88:24, out. 2017. Disponível em: <https://doi.org/10.1145/3133912>.

REMS PROJECT. *Sail: ISA specification language*. GitHub, 2026. Disponível em: <https://github.com/rems-project/sail>. Acesso em: 27 set. 2026.

ROCQ DEVELOPMENT TEAM. *Release of Rocq 9.0*. 2025. 12 mar. 2025. Disponível em: <https://rocq-prover.org/changelog/2025-03-12-rocq-9.0>. Acesso em: 27 set. 2026.

SAMMLER, M. et al. Islaris: verification of machine code against authoritative ISA semantics. In: *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. New York, NY, USA: Association for Computing Machinery, 2022. (PLDI 2022), p. 825–840. ISBN 9781450392655. Disponível em: <https://doi.org/10.1145/3519939.3523434>.

WANG, J. et al. Formalizing SPARCv8 instruction set architecture in Coq. *Science of Computer Programming*, v. 187, p. 102371, 2020. ISSN 0167-6423. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0167642318301163>.

ZELLER, A.; HILDEBRANDT, R. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, v. 28, n. 2, p. 183–200, 2002. Disponível em: <https://doi.org/10.1109/32.988498>.